

Einführung in MLOps

22 FEATURE STORE

Tobias Mérinat

teaching2025@fsck.ch

Lucerne University of
Applied Sciences and Arts

**HOCHSCHULE
LUZERN**

DEPARTMENT OF INFORMATION TECHNOLOGY
Lucerne University of Applied Sciences and Arts
6343 Rotkreuz, Switzerland

22. und 23. August 2025

Gründe für Training Serving Skew:

- Nicht repräsentative Trainingsdaten
- Falsche Cross Validation (Zeitreihen, Normalisierung über Validierungsdaten hinweg, Grouped Data)
- Daten-Drift
- Fehler bei der Neuimplementierung von Feature Pipelines
- Fehler beim Joinen von Features

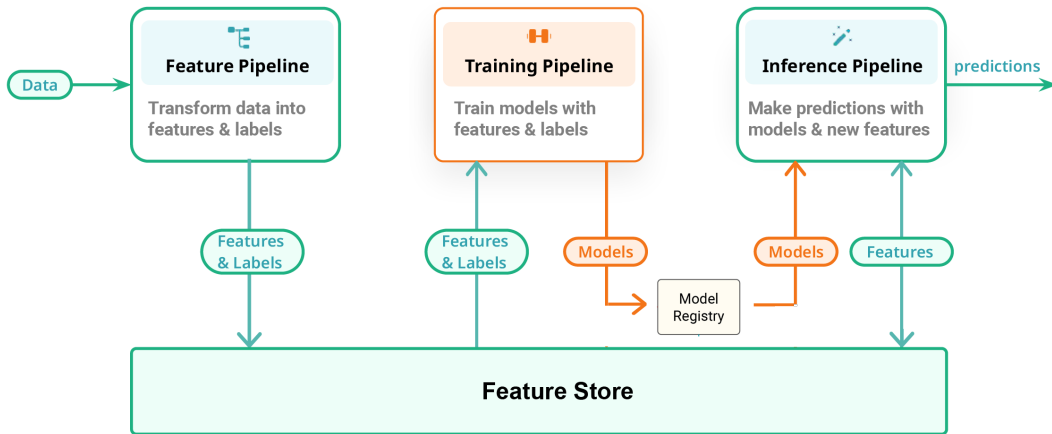
Drei Hauptaufgaben:

- 1 Latenz bei der Bereitstellung während Inferenz zu verringern
- 2 Persistieren, um historische Daten für das Training zur Verfügung zu stellen
- 3 Bereitstellung eines Katalogs, um Features durchsuchbar zu machen

Ein Feature Store muss demnach

- Features via Batch- und Streaming APIs entgegennehmen
- Features effizient speichern
- Feature Management und Discovery ermöglichen
- Features für Batch- und Online Inference zur Verfügung stellen
- Features für Training zur Verfügung stellen

Architektur mit Feature Store (hopsworks)



Für die Nutzung während der Inferenz bietet ein Feature Store zwei (bzw. drei) Zugriffsmöglichkeiten bzw. Datenbanken:

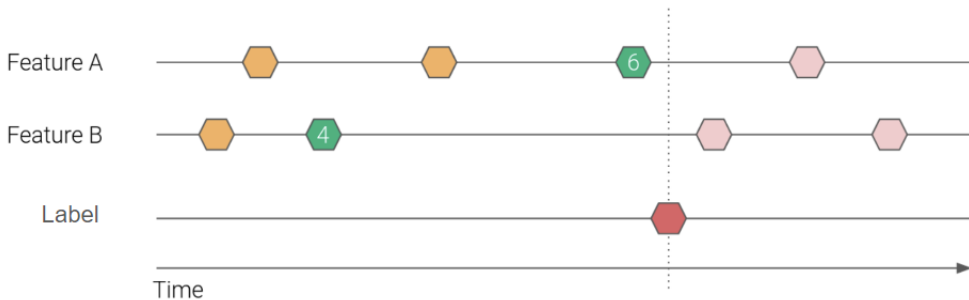
- ein Online-Store mit tiefer Latenz (Online-Prediction)
- ein Offline-Store für grosse Mengen historische Daten (Batch-Prediction)
- optional eine Vektor-Datenbank zum Speichern und Finden von Embeddings

- Backfilling und Point-in-Time Correctness
- Feature Katalog mit Discovery
- Generell: Zentrales Repository für Features mit APIs zum Schreiben, Managen und Verwenden

- Der Vorgang, aus historischen Rohdaten ein (Feature-)Datenset zu berechnen
- Also eine Feature Pipeline mit *Von/Bis* Angaben laufen zu lassen, um Features zu generieren
- Sowohl Batch-wie auch Streaming Pipelines sollten Backfilling unterstützen
- Notwendig, um Trainingdaten zu generieren (Retraining oder neues Modell)
- Notwendig, wenn ein neues Feature entwickelt oder ein bestehendes angepasst wird
- Die *gleiche* Feature Pipeline sollte für *live* Daten und für Backfilling verwendet werden
- Ohne Backfilling muss der *log and wait* Ansatz gefahren werden

- **Point-in-time Correctness:** Die Fähigkeit eines Systems, eine Berechnung genau so durchzuführen, wie sie zu einem beliebigen Zeitpunkt in der Vergangenheit stattgefunden hätte.
- **Point-in-time Correct Join:** Ein temporaler Join zwischen Tabellen, dessen Resultat den Stand beider Tabellen zu einem gegebenen Zeitpunkt widerspiegelt. Die linke Tabelle mit den Labels gibt den Zeitpunkt vor.
- Mit Point-in-time Correct Joins werden beim Zusammenführen von Tabellen keine Daten geleakt.

Point-in-Time Correct Training Data



Point-in-time Beispiel

seller_delivery_time_monthly

seller_id	event_time	avg_deliver_time_hrs
1112	oct-22	72
1112	nov-22	104
1112	dec-22	88

seller_reviews_quarterly

seller_id	event_time	avg_review_score	...
1112	oct-22	3.4	...
1112	dec-22	2.8	...

seller_delivery_time_monthly is the label Feature Group. Join on seller_id, lining up rows on event_time (point-in-time join).

seller_delivery_time_monthly_fv

seller_id	event_time	avg_deliver_time_hrs	avg_review_score	...
-----------	------------	----------------------	------------------	-----

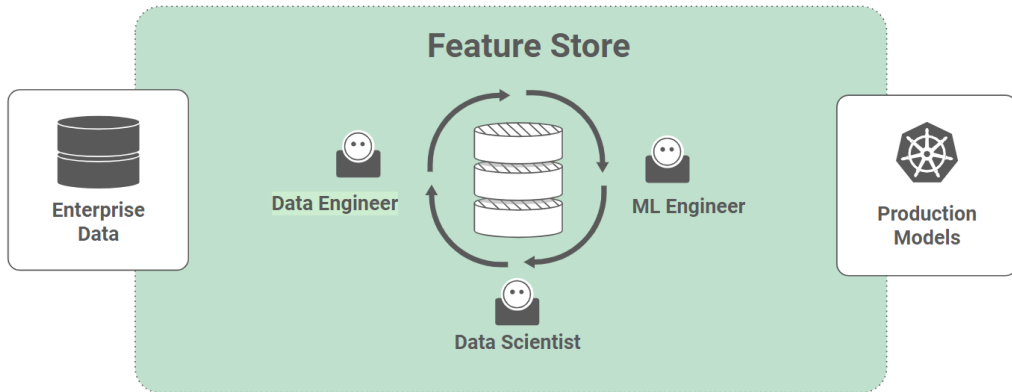
*seller_delivery_time_monthly_fv.
save_training_data(..,format="csv",..)*

seller_delivery_time_monthly_td.csv

seller_id	event_time	avg_deliver_time_hrs	avg_review_score	...
1112	oct-22	72	3.4	
1112	nov-22	104	3.4	
1112	nov-22	88	2.8	

Point-in-Time
correct JOIN

Cross-Team Collaboration with a Feature Store



Zusammenfassung der Aufgaben eine Feature Store

- Feature-Repository (Offline-Store):
 - API für Verwendung von Features (einfügen, Validierung, Inferenz, Training)
- Online-Store: Ermöglicht Anwendungsfälle, wo Inferenz-Latenz tief sein muss
- Feature-Catalog:
 - Zentrales Repository für Features, Suche, unterstützt Wiederverwendbarkeit
 - Gibt Standards für Benennung von Features, Metadaten und Dokumentation vor
- Point-in-time Correctness:
 - Das Erstellen von Trainingssets wird einfacher und weniger fehleranfällig
 - Schnellere Iterationen durch automatisiertes Backfilling (vs. Log-and-wait Ansatz)
- Data Governance:
 - Feature Lineage und Backfilling hilft bei der Reproduzierbarkeit
 - Zugriffsbeschränkungen (rollenbasierter Zugang, automatisiertes Masking/Anonymisierung, Propagieren von Beschränkungen auf abgeleitete Features)

- Hopsworks Featurestore
- Feast
- Feature Form

- Grosse Liste und Vergleiche auf featurestore.org